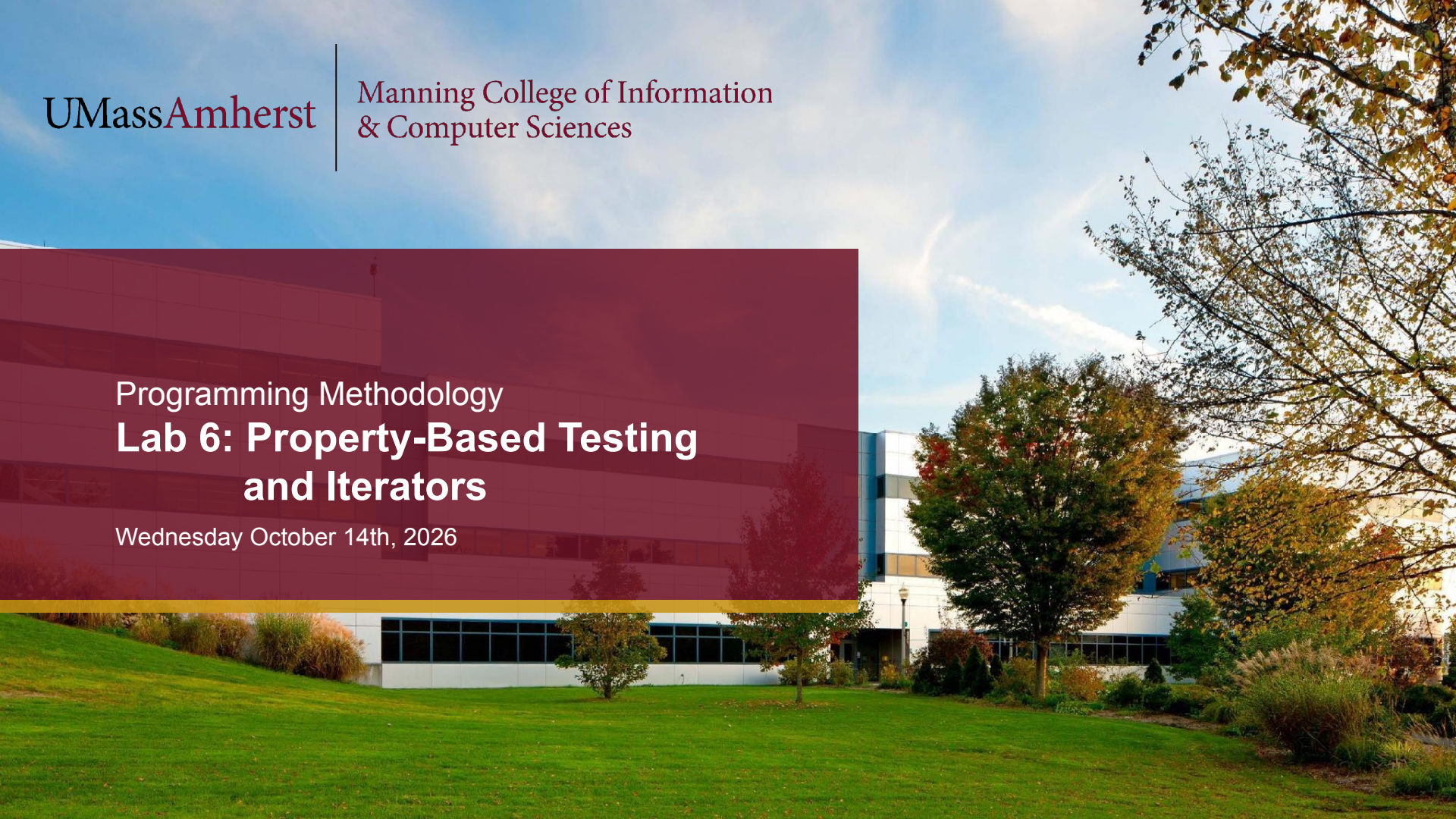


Programming Methodology

Lab 6: Property-Based Testing and Iterators

Wednesday October 14th, 2026

Weekly Lab Agenda

- Go over reminders/goals
- Review past material
- Work in groups of 3-4 to solve a few exercises.
  - Please sit with your group from last week.
- Discussion leaders will walk around and answer questions.
- Solutions to exercises will be reviewed as a class.

Reminders

- Homework 5 is due tonight (10/14) at 11:59 pm.
  - Come to office hours for help!
- If you need to miss lab for a valid and documented reason:
  - Fill out the excusal form on Canvas before the lab.
  - Waking up late or the bus being late are not excusable reasons.
- Submit what you have at the end of lab to Gradescope.
  - Lab submissions are due at 6:00 pm.
  - The submission counts towards your lab grade.

Today's Goals

- Practice writing property-based tests
- Practice with iterators

Property-Based Testing

Used when a problem has **more than one** right answer.

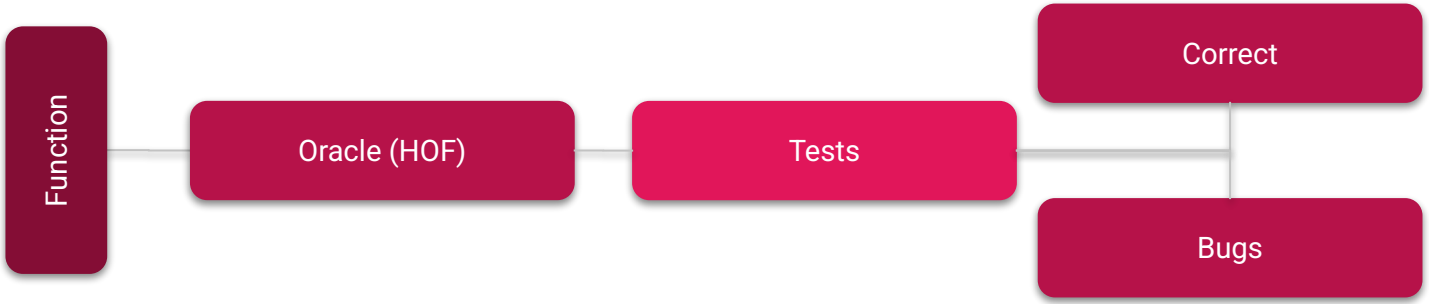
Steps for property-based testing:

1. Start with a valid input to your problem
2. Run the algorithm on that input
3. Check that the result has all necessary characteristics

Oracle Functions

In this class, **oracles** are functions that determine whether or not a function correctly solves a given problem. We can determine this by performing property based testing on the outputs of the given function for various inputs.

For example, in Homework 5, you write an oracle for the Preference Matching Problem.



Exercise 1: Permutations

A function **`genArray(n: number): number[][]`** is supposed to generate an **n×n** array of numbers such that each row and column is a permutation of the numbers from **`0`** to **`n-1`**. Assume **`n`** is nonnegative.

Write an oracle that accepts the function `genArray` as input and tests it for the described behavior.

You are given **`randomPositiveInteger`** for your convenience.

Use higher-order functions when appropriate.

Try to write your implementation in $O(n^2)$.

Solution 1: Permutations


```
function oracle(genArray: (n: number) => number[][]) {
  let n = Math.floor(Math.random() * 1000);

}
```

Solution 1: Permutations


```
function oracle(genArray: (n: number) => number[][]) {
  let n = Math.floor(Math.random() * 1000);
  let a = genArray(n);

}
```

Solution 1: Permutations


```
function oracle(genArray: (n: number) => number[][]) {
  let n = Math.floor(Math.random() * 1000);
  let a = genArray(n);
  const valid = x => x % 1 === 0 && 0 <= x && x < n;

}
```

Solution 1: Permutations


```
function oracle(genArray: (n: number) => number[][]) {
  let n = Math.floor(Math.random() * 1000);
  let a = genArray(n);
  const valid = x => x % 1 === 0 && 0 <= x && x < n;
  function isPerm(p: number[]) {

  }
}
```

Solution 1: Permutations


```
function oracle(genArray: (n: number) => number[][]) {
  let n = Math.floor(Math.random() * 1000);
  let a = genArray(n);
  const valid = x => x % 1 === 0 && 0 <= x && x < n;
  function isPerm(p: number[]) {
    assert(p.length === n, 'length is n');

  }
}
```

Solution 1: Permutations

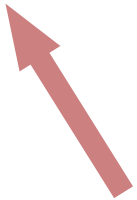

```
function oracle(genArray: (n: number) => number[][]) {
  let n = Math.floor(Math.random() * 1000);
  let a = genArray(n);
  const valid = x => x % 1 === 0 && 0 <= x && x < n;
  function isPerm(p: number[]) {
    assert(p.length === n, 'length is n');
    assert(p.every(valid),'valid numbers');

  }
}
```

Solution 1: Permutations

```
function oracle(genArray: (n: number) => number[][]) {
  let n = Math.floor(Math.random() * 1000);
  let a = genArray(n);
  const valid = x => x % 1 === 0 && 0 <= x && x < n;
  function isPerm(p: number[]) {
    assert(p.length === n, 'length is n');
    assert(p.every(valid),'valid numbers');
    const f = Array(n).fill(1);
    assert(p.every(e => --f[e] === 0), 'no duplicates');

  }
}
```



Note: We're checking for duplicates in a single linear scan which takes O(**n**) time. This approach works for **n = 0** and an empty array.

Solution 1: Permutations


```
function oracle(genArray: (n: number) => number[][]) {
  let n = Math.floor(Math.random() * 1000);
  let a = genArray(n);
  const valid = x => x % 1 === 0 && 0 <= x && x < n;
  function isPerm(p: number[]) {
    assert(p.length === n, 'length is n');
    assert(p.every(valid),'valid numbers');
    const f = Array(n).fill(1);
    assert(p.every(e => --f[e] === 0), 'no duplicates');
  }
  assert(a.length === n, 'array has n rows');

}
```

Solution 1: Permutations

```
function oracle(genArray: (n: number) => number[][]) {
  let n = Math.floor(Math.random() * 1000);
  let a = genArray(n);
  const valid = x => x % 1 === 0 && 0 <= x && x < n;
  function isPerm(p: number[]) {
    assert(p.length === n, 'length is n');
    assert(p.every(valid),'valid numbers');
    const f = Array(n).fill(1);
    assert(p.every(e => --f[e] === 0), 'no duplicates');
  }
  assert(a.length === n, 'array has n rows');
  a.forEach(isPerm); // each row is a permutation

}
```



a has length **n**, and we call **isPerm** (an O(**n**) function) for each element of **a**. This will take O($\mathbf{n}^2$) time.

Solution 1: Permutations

```
function oracle(genArray: (n: number) => number[][]) {
  let n = Math.floor(Math.random() * 1000);
  let a = genArray(n);
  const valid = x => x % 1 === 0 && 0 <= x && x < n;
  function isPerm(p: number[]) {
    assert(p.length === n, 'length is n');
    assert(p.every(valid),'valid numbers');
    const f = Array(n).fill(1);
    assert(p.every(e => --f[e] === 0), 'no duplicates');
  }
  assert(a.length === n, 'array has n rows');
  a.forEach(isPerm); // each row is a permutation
  for (let k = 0; k < n; ++k) { isPerm(a.map(r => r[k])); }
     // each column is a permutation
}
```

Again, this will take $O(n^2)$ time.

Note: A different approach, where we check directly that every number from **0** to **n - 1** is included requires a linear scan for each number. That would be $O(n^2)$ per row/column and $O(n^3)$ overall. Do not do this!

Iterators Review

```
interface Iterator<ValueType> {
  next(): IteratorResult<ValueType>;
}

interface IteratorResult<ValueType> {
  done: boolean; // true indicates Iterator has reached its end
  value?: ValueType; // could be undefined, specifically if done is true
}
```

- These interfaces are built into JS/TS.
- Conceptually, an iterator is an object that provides access to items in some collection following some order.
- In JS/TS, the **`next()`** method of an iterator yields the next item and moves the iterator forward.
  - **`next()`** returns an **`IteratorResult`**, an object specifying the resulting item, **`value`**, and whether the iterator has completed, **`done`**.

Iterators Review

```
interface Iterable<ValueType> {
  [Symbol.iterator](): Iterator<ValueType>;
}

interface IterableIterator<ValueType> {
  next(): IteratorResult<ValueType>;
  [Symbol.iterator](): Iterator<ValueType>;
}
```

- **`Iterable`** is a built-in interface in JS/TS describing objects that can be iterated through.
  - i.e. using a range based (**`for... of...`**) loop
- **`[Symbol.iterator]()`** is a special method that returns an iterator for the iterable.
- **`IterableIterator`** is a built-in interface equivalent to implementing both the **`Iterable`** and **`Iterator`** interfaces.
  - i.e. **`implements Iterable<ValueType>, Iterator<ValueType>`**

Iterators Review

The class **`MyIterator`** is an example that follows the **`Iterator`** and **`Iterable`** interfaces.

The **`IterableIterator`** interface is equivalent to using **`implements Iterable<T>, Iterator<T>`**.

This iterator just counts and never terminates.

```
class MyIterator implements IterableIterator<number> {
  private _num: number;

  constructor(n: number) {
    this._num = n;
  }

  next(): IteratorResult<number> {
    return { done: false, value: this._num++ };
  }

  [Symbol.iterator]() {
    return this;
  }
}
```

Exercise 2: Iterators

Write a class **`AlternatingIterator<T>`** that takes two iterables and yields their elements alternately, starting with the first iterable.

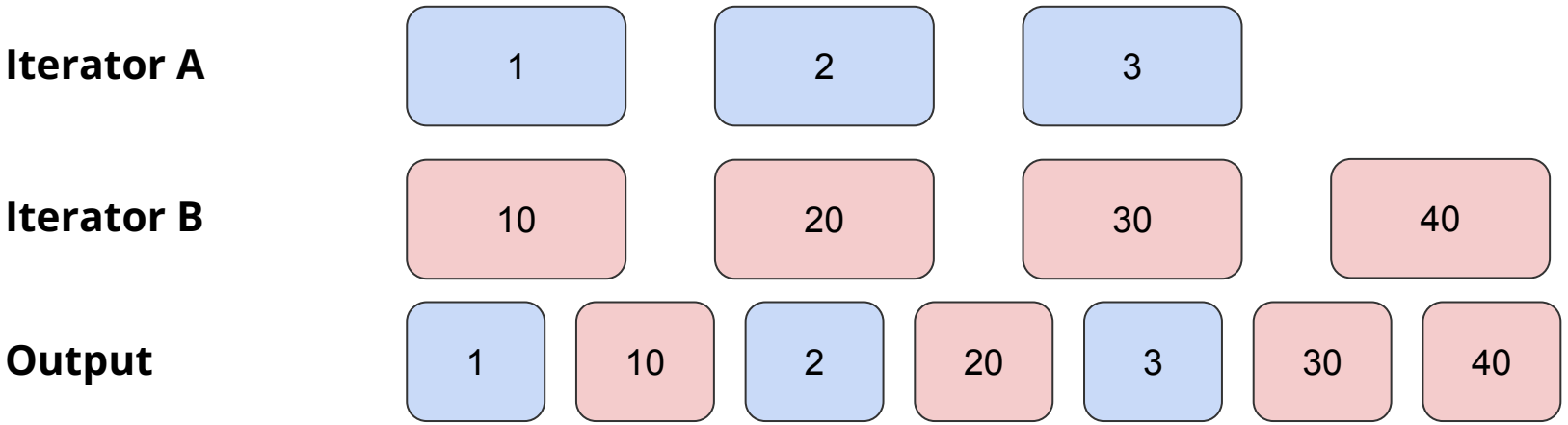
- When one iterator is exhausted, continue yielding elements from the other.
- The iterator finishes when both iterators are exhausted.

The class should implement **`IterableIterator<T>`** so that it can be used in a **`for...of`** loop.

```
class AlternatingIterator<T> implements IterableIterator<T> {
  // TODO: Implement this class
}
```

Exercise 2: Iterators (Diagram)

Alternating order:



Solution 2: Iterators

```
class AlternatingIterator<T> implements IterableIterator<T> {

  private first: Iterator<T>;
  private second: Iterator<T>;
  private useFirst: boolean;

  // ...
}
```

We store the two iterators and a boolean indicating which iterator should be used next.

Solution 2: Iterators

```
class AlternatingIterator<T> implements IterableIterator<T> {
  private first: Iterator<T>;
  private second: Iterator<T>;
  private useFirst: boolean;

  constructor(a: Iterable<T>, b: Iterable<T>) {
    this.first = a[Symbol.iterator]();
    this.second = b[Symbol.iterator]();
    this.useFirst = true;
  }

  [Symbol.iterator](): IterableIterator<T> {
    return this;
  }
}
```

We obtain an iterator from each iterable using **`[Symbol.iterator]()`**.
We initialize **`useFirst`** to **`true`** so the first iterator is selected initially.

Solution 2: Iterators

```
class AlternatingIterator<T> implements IterableIterator<T> {
  // ...
  next(): IteratorResult<T> {
    const curr = this.useFirst ? this.first : this.second;
    const other = this.useFirst ? this.second : this.first;

    const result = curr.next();

  }
}
```

We use the boolean variable **`useFirst`** to decide which iterator to access next.

Solution 2: Iterators

```
class AlternatingIterator<T> implements IterableIterator<T> {
  // ...
  next(): IteratorResult<T> {
    const curr = this.useFirst ? this.first : this.second;
    const other = this.useFirst ? this.second : this.first;

    const result = curr.next();

    if (!result.done) {
      this.useFirst = !this.useFirst;
      return result;
    }

  }
}
```

If the selected iterator produces a value, we return it and switch to the other iterator for the next call.

Solution 2: Iterators

```
class AlternatingIterator<T> implements IterableIterator<T> {
  // ...
  next(): IteratorResult<T> {
    const curr = this.useFirst ? this.first : this.second;
    const other = this.useFirst ? this.second : this.first;

    const result = curr.next();

    if (!result.done) {
      this.useFirst = !this.useFirst;
      return result;
    }

    return other.next();
  }
}
```

If the selected iterator is exhausted, we try the other iterator.
The combined iterator is finished when both iterators are exhausted.

Solution 2: Iterators

```
const it = new AlternatingIterator(
  [1, 2, 3],
  [10, 20, 30, 40]
);

it.next(); // { done: false, value: 1 }
it.next(); // { done: false, value: 10 }
it.next(); // { done: false, value: 2 }
it.next(); // { done: false, value: 20 }
it.next(); // { done: false, value: 3 }
it.next(); // { done: false, value: 30 }
it.next(); // { done: false, value: 40 }
it.next(); // { done: true, value: undefined }
```

The iterator alternates between the two inputs, then continues with the remaining elements until both are exhausted.